Electronics and
Computer Science

UNIVERSITY OF
Southampton

7. SystemVerilog

Electronics and
Computer Science

UNIVERSITY OF
Southampton

7.1: Data Types and Literals

Integer data types

Name	Size and values	Example
bit	single bit, 2 values: 0,1	bit [3:0] HighNibble;
byte	8 bits, signed	byte a, b;
shortint	16 bits, signed	shortint c, d;
int	32 bits, signed	int i,j;
longint	64 bits,signed	longint lword;
logic	single bit, 4 values: 0,1,X,Z	
reg	identical to logic	
integer	32 bits, 4 values: 0,1,X,Z	
wire	single bit, 4 values 0,1,X,Z and 8 strength levels	wire z1,z2;

NB use mainly **logic** for synthesis

(There are more net data types like wire inherited from Verilog)

Unsigned data types

integer data types can be declared as signed or unsigned.

byte, shortint, int, integer, and longint default to signed.
bit, reg, and logic default to unsigned, as do arrays of these types.

Name	Size and values	Range
byte signed	8 bits, signed	-128.. +127
byte unsigned	8 bits, unsigned	0..255
shortint	16 bits, signed	-32768 ... 32767
shortint unsigned	16 bits, unsigned	0 ... 65536;
longint	64 bits,signed	-2^63 ... 2^63-1
longint unsigned	64 bits,unsigned	0 ... 2^64-1

Arrays

Examples:

```
logic [7:0]  v;       // an 8-element packed logic vector
bit [3:0][7:0]  memory;  // a 2-dimensional bit array

int  x[63:0]; // if dimensions follow name, array is 'unpacked'

logic [7:0] register [0:15];
// this array has packed and unpacked dimensions
// use this to model memory (see previous lectute)
```

Enumerated and user defined data types

Examples:

```
enum {Monday, Tuesday, Wednesday, Thursday, Friday} WeekDay;

typedef enum {idle, cycle1, cycle2 } state;

// state is a user-defined data type

state present,next;

enum {a=0, b=7, c=5, d=8} myEnum;

// user can assign values to enum constants
```

literals

General syntax: [*size*['*base*]] *value*

Examples:

1'b0 - binary 0
4'hF - hex base, binary equivalent: 1111
10 - decimal base (default)
'o6 - octal base, binary equivalent: 110

size is the number of bits, default is 32 bits

'*base* represents the radix, default is decimal

Base	Symbol	Legal values
binary	b,B	0,1,x,X,z,Z,?,_
octal	o,O	0-7, x,X,z,Z,?,_
decimal	d,D	0-9, x,X,z,Z,?,_
hexadecimal	h,H	0-9,a-f,A-F,x,X,z,Z,?,_

? is the same as z or Z
_ (underscore) is ignored, it is used to enhance legibility of long literals

Electronics and
Computer Science

UNIVERSITY OF
Southampton

7.2: Hierarchy

Parameters

Example of module declaration with a parameter

```
module cpu #(parameter Dsize = 8) // data bus width
(input logic clk,
  input logic reset, // master reset
  output logic[Dsize-1:0] outport // output port, parameterised width
);
```

```
//instantiation example (for example in a testbench)
cpu  #(.Dsize(8)) c0 (.*);  // create an 8-bit instance of cpu
                            // note implicit port mapping  .*
```

We can also use parameters in signal declarations:

```
parameter Psize = 6;
```

Port mapping

- named port mapping
  - use where implicit mapping cannot be applied
- implicit port mapping
  - 'magic' SystemVerilog shortcut: .*
- positional (ordered) port mapping
  - inherited from Verilog; avoid

named port mapping

```
prog #(.Psize(Psize),.Isize(Isize))
     progMemory (.address(ProgAddress),.I(I));

// ports are named and explicitly mapped to local signals
// positions  are arbitrary, i.e. order in which ports are listed
// does not matter
```

Named mapping **may** always be used in preference to other mapping styles, but it **must** be used in the following cases:

(1)the port and connecting net sizes do not match, e.g. (…, .A(A[15:8]), …)

(2)the port is unconnected, e.g.: (…, .bus(), …)

positional port mapping – inherited from Verilog

```
prog #(Psize, Isize)
     progMemory (ProgAddress, I);

// Connecting nets ProgAddress, I must be listed in the same
// order as corresponding port declarations. Same for parameters
//
// Local nets and corresponding ports  may have  different names but
// must have the same types and sizes
```

Ideally, avoid positional port mapping and **use named** or implicit **mapping** instead. Named mapping is less prone to errors and adds legibility to the code.

Implicit port mapping

```
cpu co (.*);

// local logic signal declarations and corresponding ports must
// have same names, types and sizes
```

There are six important rules for implicit port connections:

(1) named and implicit ports (.*) may be mixed in the same instantiation, but there must be only one implicit item,
(2) positional and implicit ports (.*) may be mixed in the same instantiation, but there must be only one implicit item,
(3) implicit mapping may not be used if the port and connecting net sizes do not match,
(4) implicit mapping may not be used if the port and connecting net have different names,
(5) implicit mapping may not be used if the port is unconnected,
(6) all nets or variables connected to the implicit ports must be declared in the instantiating module, either as explicit local declarations or as explicit port declarations.

Electronics and
Computer Science

UNIVERSITY OF
Southampton

7.3: Time specifications

time literals, timescale directive

time literals can have integer or fixed-point format:
0.1ns
40ps

literals in simulation time delays may specify time without a time unit:
#5 …
#2 …

The time unit is defined in a timescale directive:
`timescale *time_resolution / precision*

```
`timescale 1ns / 10ps
…
#5;  // wait 5ns

#0.01;    // wait for 0.01*time_resolution (10ps)
#0.001;  // this is below time resolution hence it will produce zero delay
```

SystemVerilog timeunit, timeprecision

- Instead of a directive, can use declarations in SystemVerilog:

```
timeunit 1ns;
timeprecision 100ps;
```

- Better because the scope is defined. Timescale directive applies across modules.
- Precision must be less than or equal to time unit. Ratio of 1:10 or 1:100 is common
- Some simulators (e.g. ModelSim) insist that, if used in one, timeunit, timeprecision must be in every module. (Watch out for this!)

Electronics and
Computer Science

UNIVERSITY OF
Southampton

7.4: Loops

Loop statements

All used in always or initial blocks

- for
- while
- do – while
- repeat
- foreach
- forever

generate for is structural and used outside always/initial block

- See example later in this lecture

for loop

for (*initialisation_expression*; *condition_expresion*; *step_expression*)
statement;

initialisation_expression executes once at the start;

condition_expression executes at the start of each iteration; if its boolean value is not true (e.g. false, x, or z), the loop exits; otherwise statement executes;

step_expression executes at the end of each iteration.

for loop execution order:

initialisation_expression
first iteration:
    condition_expression
    statement
    step_expression
2nd iteration:
    condition_expression
    statement
    step_expression
etc…

while loop

while (expression) *statement*;

expression is evaluated at the start of each iteration.
If it returns true, *statement* executes, otherwise the loop exits.

Also do…while, like C

```
int i=0;
while(i<10)
begin
      #5 $display("i = %d\n",i);
           i++;
end
```

repeat loop

repeat (*expression*) *statement*;

The **repeat** loop is similar to **for** loop but has no loop control variable; *statement* is evaluated *expression* times.

```
int i=0;
repeat (16)
  $display("i = %d", i++);
```

is equivalent to:

```
for(int i=0;i<16;i++)
    $display("i = %d",i);
```

foreach loop

foreach (*array_identifier* [*loop_variables*])
    statement;

foreach loop iterates over the elements of an array.
Its argument is an identifier that designates an array followed by a list of loop variables enclosed in square brackets.
Each loop variable corresponds to one of the dimensions of the array.

```
string words [1:2] = { "hello", "world" };  // array of strings
foreach( words [ j ] )
     $display( j , words[j] ); // print each index and value
```

```
logic mem [0:63] [7:0];
foreach( mem[ address, word ] )
     mem[address][word] = 1'b0; // initialize
```

forever loop

forever *statement*;

forever loop continuously repeats its *statement*.
Only use in procedural blocks and with timing controls, otherwise it will hang the simulation.

```
initial
begin
    clk=0;
    forever #10 clk = ~clk;
end
```